

Spring Boot Web Development, Validation & RFC 7807

Chapter 3 — Pro Spring Boot 4 (4th ed) · `chapters/chapter-03-web-development/customer`

Spring Boot 4.0.6

Spring Framework 7.0.7

Servlet 6.1 / Jakarta EE 11

Jackson 3 + XML

Jakarta Validation 3.1

RFC 7807 ProblemDetail

Contents

1. [What this chapter covers](#)
2. [Code walkthrough](#)
3. [Mermaid diagrams](#)
4. [Live run & endpoint evidence](#)
5. [Session context](#)

1 · What this chapter covers

Chapter 3 evolves the simple CRM API into a production-shaped service. The concrete capabilities demonstrated in this codebase are:

- **REST CRUD** with proper HTTP semantics (201 + `Location`, 404, 204).
- **Declarative validation** (Jakarta Validation 3.1) on the request body via `@Valid`.
- **RFC 7807** `ProblemDetail` error contract through a `@RestControllerAdvice`.
- **Content negotiation** — JSON and XML from the same handler (Jackson XML on the classpath).
- **Path-based API versioning** — a legacy `/v1_1` and a strict `/v2_0` alongside the main `/v1`.
- **Declarative HTTP client** — `@HttpExchange` + `@ImportHttpServices` (a Spring Framework 7 feature).

2 · Code walkthrough

Domain model with constraints

Customer.java:8-24

An immutable Java `record` carrying Jakarta Validation annotations: `@NotBlank` on `name`, `@Email` + `@NotBlank` on `email`, and a `@Pattern` regex on `phone`. A secondary constructor defaults `id` to `null` so new entities can be created without one.

REST controller & HTTP semantics

CustomerController.java

`@RestController` mapped to `/api/v1/customers`. `create()` and `update()` take `@Valid @RequestBody Customer`, so an invalid payload throws `MethodArgumentNotValidException` before the method body runs. `create()` builds the `Location` header with `ServletUriComponentsBuilder` and returns `201 Created`; `findById()` returns `404` when absent; `delete()` returns `204 No Content`.

Global exception handling → RFC 7807

GlobalExceptionHandler.java

A `@RestControllerAdvice` centralizes error mapping. On `MethodArgumentNotValidException` it builds `ProblemDetail.forStatusAndDetail(400, "Validation failed")`, sets a human `title`, and attaches a flattened `errors` property of `field: message` pairs. A second handler maps `NoResourceFoundException` to a `404` `ProblemDetail` with a custom `type` URI. This is the standardized error contract — no raw stack traces leak to clients.

Content negotiation (JSON / XML)

`pom.xml:42-45`

Adding `jackson-dataformat-xml` registers an XML `HttpMessageConverter`. The same controller method serves JSON or XML purely based on the request `Accept` header — no branching in code.

Path-based API versioning

`v1/CustomerControllerV1.java` · `v2/CustomerControllerV2.java`

Two extra controllers demonstrate contract evolution without breaking clients: `/api/v1_1/customers` is legacy (no `@Valid`, accepts a partial body) while `/api/v2_0/customers` is strict (`CustomerV2` requires `title`, `name`, and a valid `email`). The same `GlobalExceptionHandler` serves both.

Declarative HTTP client

`client/CurrencyClient.java` · `client/ClientConfig.java`

`CurrencyClient` is an interface annotated `@HttpExchange("https://open.er-api.com/v6/latest")` with a `@GetExchange("/{base}")` method — Spring generates the implementation.

`@ImportHttpServices(CurrencyClient.class)` registers it as a bean (Spring Framework 7). It is exercised in tests rather than wired into a controller.

In-memory repository & startup seeding

`CustomerRepository.java` · `CustomerConfiguration.java`

A `ConcurrentHashMap`-backed `Repository` keeps the demo self-contained. `CustomerConfiguration` seeds three customers on `ApplicationReadyEvent` (the same lifecycle hook as Chapter 2).

3 · Mermaid diagrams

Rendered from Mermaid and embedded as self-contained images (they also print in the PDF export). Expand each for the re-renderable source.

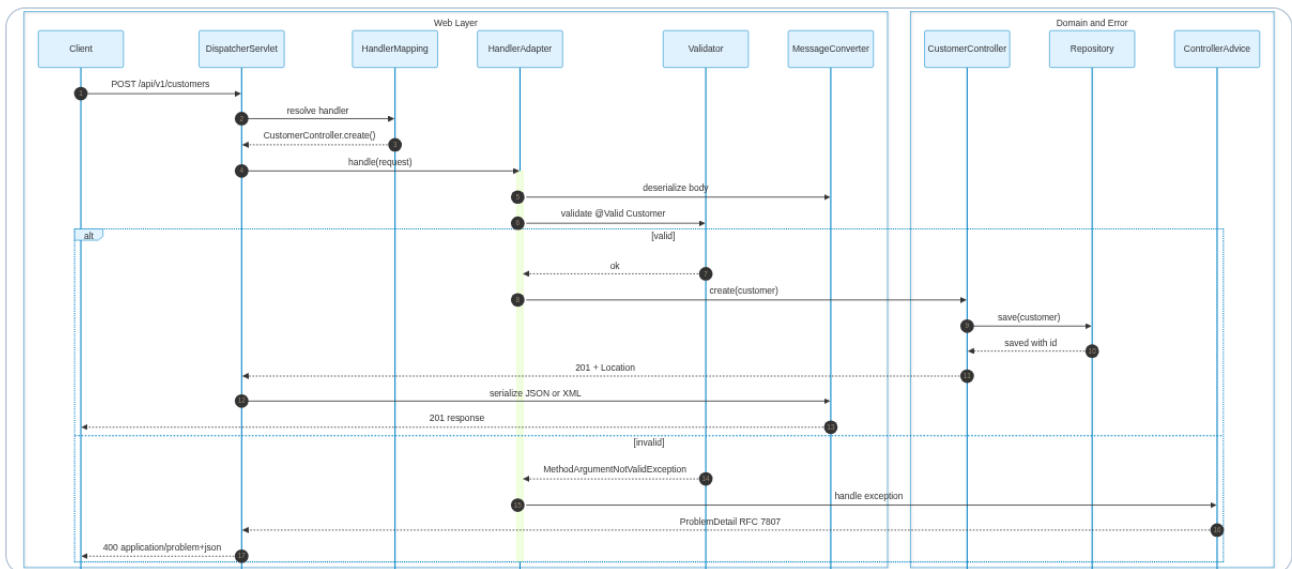


Fig 1 — Spring MVC request lifecycle. A `POST` flows through `DispatcherServlet` → handler resolution → validation → controller, then either a `201` response or an exception routed to the advice for a `400 ProblemDetail`.

[Mermaid source](#)

```
%%{init: {'theme':'base','themeVariables':{'primaryColor':'#e0f2fe','primaryBorderColor':'#0284c7','line
```

sequenceDiagram

```
    autonumber
```

```
    box Web Layer
```

```
        participant C as Client
```

```
        participant DS as DispatcherServlet
```

```
        participant HM as HandlerMapping
```

```
        participant AD as HandlerAdapter
```

```
        participant VAL as Validator
```

```
        participant JM as MessageConverter
```

```
    end
```

```
    box Domain and Error
```

```
        participant CT as CustomerController
```

```
        participant REPO as Repository
```

```
        participant ADV as ControllerAdvice
```

```
    end
```

```
C->>DS: POST /api/v1/customers
```

```
DS->>HM: resolve handler
```

```
HM-->>DS: CustomerController.create()
```

```
DS->>AD: handle(request)
```

```
activate AD
```

```
AD->>JM: deserialize body
```

```
AD->>VAL: validate @Valid Customer
```

```
alt valid
```

```
    VAL-->>AD: ok
```

```
    AD->>CT: create(customer)
```

```
    CT->>REPO: save(customer)
```

```
    REPO-->>CT: saved with id
```

```
    CT-->>DS: 201 + Location
```

```
    DS->>JM: serialize JSON or XML
```

```
    JM-->>C: 201 response
```

```
else invalid
```

```
    VAL-->>AD: MethodArgumentNotValidException
```

```
    AD->>ADV: handle exception
```

```
    ADV-->>DS: ProblemDetail RFC 7807
```

```
    DS-->>C: 400 application/problem+json
```

```
end
```

```
deactivate AD
```

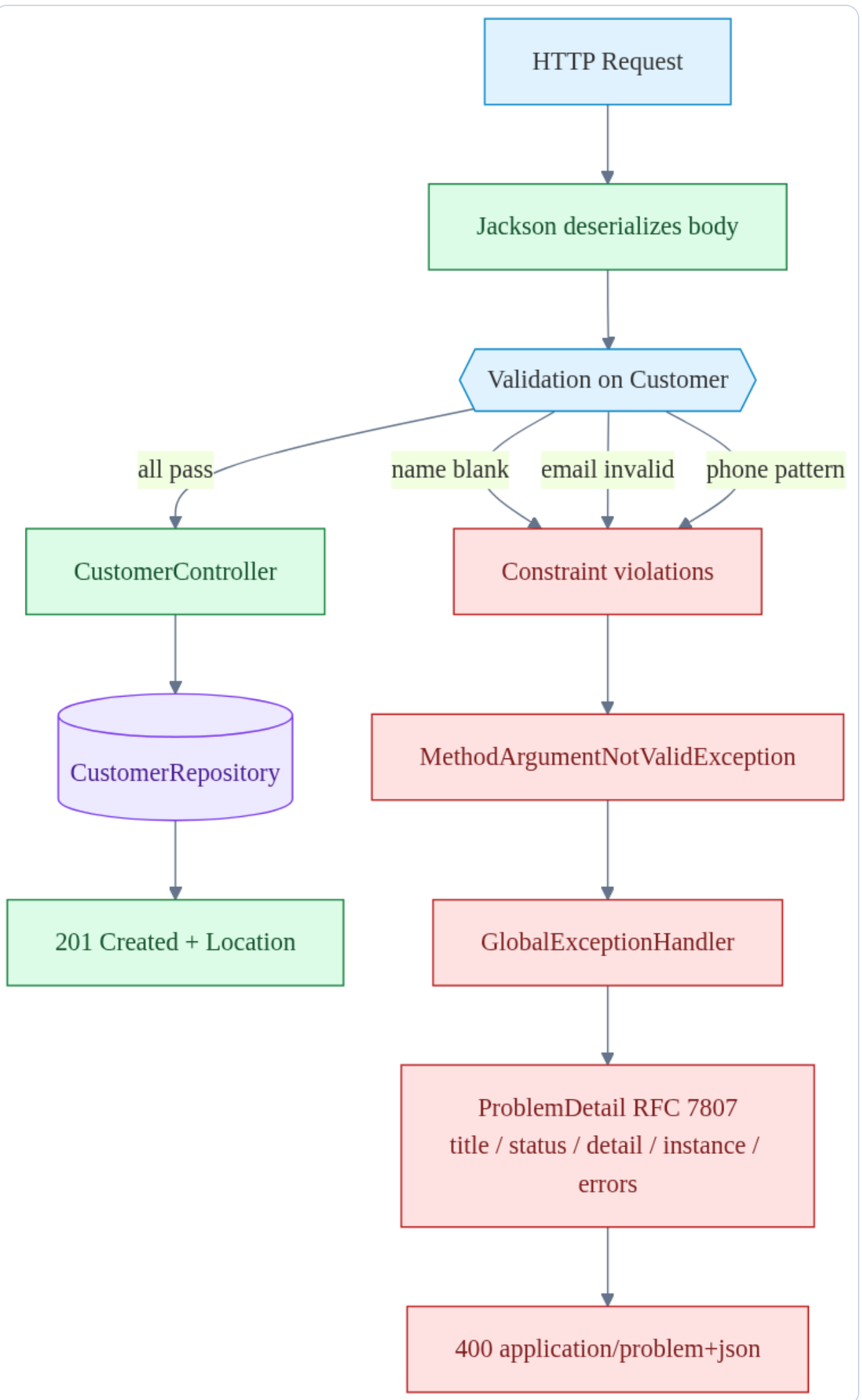


Fig 2 — Validation → RFC 7807 flow. Constraint violations on `Customer` become a `MethodArgumentNotValidException`, handled by `GlobalExceptionHandler` and returned as a standardized `ProblemDetail`.

[Mermaid source](#)

```
%%{init: {'theme':'base','themeVariables':{'primaryColor':'#e0f2fe','primaryBorderColor':'#0284c7','lineColor':'#0284c7'}}}
flowchart TD
    REQ["HTTP Request"] --> MC["Jackson deserializes body"]
    MC --> V["Validation on Customer"]
    V -->|name blank| FAIL["Constraint violations"]
    V -->|email invalid| FAIL
    V -->|phone pattern| FAIL
    V -->|all pass| CTRL["CustomerController"]
    CTRL --> REPO["CustomerRepository"]
    REPO --> RESP["201 Created + Location"]
    FAIL --> EX["MethodArgumentNotValidException"]
    EX --> ADV["GlobalExceptionHandler"]
    ADV --> PD["ProblemDetail RFC 7807<br/>title / status / detail / instance / errors"]
    PD --> ERR["400 application/problem+json"]
    classDef ok fill:#dcfce7,stroke:#15803d,color:#14532d;
    classDef err fill:#fee2e2,stroke:#b91c1c,color:#7f1d1d;
    classDef store fill:#ede9fe,stroke:#7c3aed,color:#4c1d95;
    class RESP,MC,CTRL ok;
    class FAIL,EX,ADV,PD,ERR err;
    class REPO store;
```

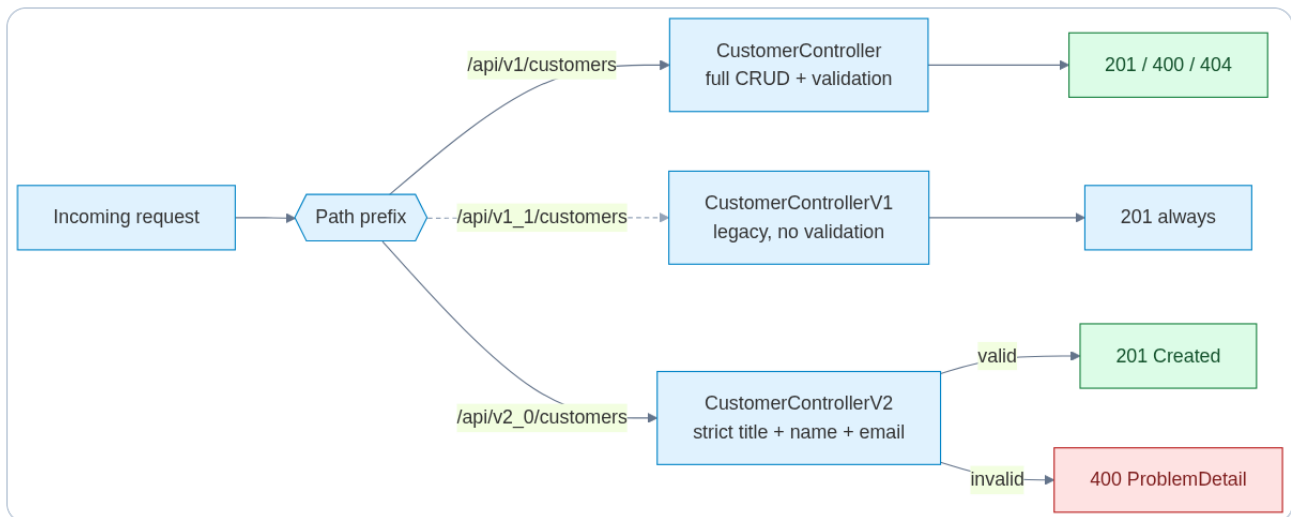


Fig 3 — Path-based API versioning. The path prefix selects the controller; the legacy `v1_1` accepts partial bodies while the strict `v2_0` enforces validation.

[Mermaid source](#)

```

%%{init: {'theme':'base','themeVariables':{'primaryColor':'#e0f2fe','primaryBorderColor':'#0284c7','line
flowchart LR
    R["Incoming request"] --> P{"Path prefix"}
    P -->|"/api/v1/customers"| V1["CustomerController<br/>full CRUD + validation"]
    P -.->|"/api/v1_1/customers"| L["CustomerControllerV1<br/>legacy, no validation"]
    P -->|"/api/v2_0/customers"| M["CustomerControllerV2<br/>strict title + name + email"]
    V1 --> Vout["201 / 400 / 404"]
    L --> Lout["201 always"]
    M -->|valid| Mout["201 Created"]
    M -->|invalid| PDErr["400 ProblemDetail"]
    classDef ok fill:#dcfce7,stroke:#15803d,color:#14532d;
    classDef err fill:#fee2e2,stroke:#b91c1c,color:#7f1d1d;
    class Mout,Vout ok;
    class PDErr err;
    linkStyle 2 stroke:#94a3b8,stroke-dasharray:4 3;
  
```

4 · Live run & endpoint evidence

Produced by running `mvn spring-boot:run` and calling the endpoints with `curl`.

Startup (key lines)

```
Tomcat initialized with port 8080 (http)
Root WebApplicationContext: initialization completed in 4945 ms
Tomcat started on port 8080 (http) with context path '/'
Started CustomerApplication in 9.7 seconds (process running for 11.329)
# ApplicationReadyEvent seed (CustomerConfiguration):
Customer[id=..., name=Jane, email=jane@example.com, phone=1-800-PHONE]
Customer[id=..., name=Jack, email=jack@example.com, phone=1-800-PHONE]
Customer[id=..., name=John, email=john@example.com, phone=1-800-PHONE]
```

Endpoint results

Request	Result	What it demonstrates
GET /api/v1/customers (Accept: json)	200	JSON via Jackson message converter
GET /api/v1/customers (Accept: xml)	200	Content negotiation → XML, same handler
POST valid body	201 + Location	ServletUriComponentsBuilder location header
POST invalid body	400	ProblemDetail with errors property
GET /api/nope	404	ProblemDetail with custom type URI
POST /api/v1_1/customers (no email)	201	Legacy version accepts partial body
POST /api/v2_0/customers (no title/email)	400	Strict version enforces validation

Sample responses (verbatim)

```
# GET .../customers (Accept: application/xml) -> 200
<ValuesView><item><id>2df26b25-...</id><name>Jane</name>
<email>jane@example.com</email><phone>1-800-PHONE</phone></item>...</ValuesView>

# POST /api/v1/customers (valid) -> 201, headers:
Location: http://localhost:8080/api/v1/customers/bc883004-6629-4086-81ee-dd93a2a4dae7
{"id":"bc883004-...", "name":"Alice", "email":"alice@example.com", "phone":"1-800-5551"}

# POST /api/v1/customers (invalid) -> 400 application/problem+json
{"detail":"Validation failed","instance":"/api/v1/customers","status":400,
 "title":"Invalid Request Content",
 "errors":"name: Name cannot be empty, phone: Invalid phone number, email: Invalid email format"}

# GET /api/nope -> 404 application/problem+json
{"detail":"The requested resource was not found","instance":"/api/nope","status":404,
 "title":"Resource Not Found","type":"https://api.crm.com/errors/not-found"}

# POST /api/v1_1/customers {"name":"Legacy"} -> 201 (legacy: email null allowed)
{"id":"b3a36cdd-...", "name":"Legacy", "email":null}

# POST /api/v2_0/customers {"name":"Modern"} -> 400 (strict)
{"detail":"Validation failed","instance":"/api/v2_0/customers","status":400,
 "title":"Invalid Request Content","errors":"email: must not be blank, title: Title is required"}
```

5 · Session context

This document is the record of the working session that produced it.

Request

“explain code, run app and create mermaid diagrams in html file. session context should be captured in same html file” — for `chapters/chapter-03-web-development/`.

Steps performed

1. **Explored** the module: `README.md`, `TECHNICAL_DEEP_DIVE.md`, and 15 main/test source files under `customer/src`.
2. **Explained** the code (Section 2), grounded in file/line references.
3. **Ran** the app with `mvn spring-boot:run` (started in 9.7s), then exercised every endpoint with `curl` to capture real evidence (Section 4).
4. **Generated** three Mermaid diagrams (MVC lifecycle, validation/RFC 7807, versioning), rendered to PNG and embedded as data URIs (Section 3).
5. **Authored** this self-contained HTML report.

Environment & tooling

- Runtime: Java 25 (project targets Java 21); Spring Boot 4.0.6 / Spring Framework 7.0.7; Tomcat 11; port 8080.
- Diagrams rendered via `mermaid.ink` (server-side) and embedded as base64 data URIs — the HTML has **no external scripts, CSS, or image files**.
- Notes: Mermaid rejects `@` in flowchart edge labels and `:` in some quoted labels — sources above use sanitized labels that match the images.

Reproduce

```
cd chapters/chapter-03-web-development/customer
mvn spring-boot:run
# then:
curl -H "Accept: application/xml" http://localhost:8080/api/v1/customers
curl -X POST http://localhost:8080/api/v1/customers \
  -H "Content-Type: application/json" \
  -d '{"name":"","email":"bad","phone":"x"}' # -> 400 ProblemDetail
```

Chapter takeaways. Standardize error contracts with RFC 7807 so microservices don't drift; validate at the boundary with `@Valid` and handle violations centrally via `@RestControllerAdvice`; version by path to evolve contracts safely; and let content negotiation, not code branches, pick JSON vs XML.

Generated from `chapters/chapter-03-web-development/` · source at `.../customer/src/main/java/com/apress/crm/customer/`. Self-contained (no external resources). Companion to the Chapter 2 ApplicationContext lifecycle report.